

## Table des matières

|                                              |   |
|----------------------------------------------|---|
| Cours 07 — CI/CD et qualité analytique ..... | 1 |
| Objectifs .....                              | 1 |
| Théorie .....                                | 2 |
| Points clés à retenir.....                   | 7 |
| Exercices.....                               | 7 |

## Cours 07 — CI/CD et qualité analytique

*Théorie · Module 7/8 — Formation Analytics Engineering · ~25 min · avancé*

Support d'avant la refonte. La formation a été refondue le 2026-09-20, de Snowflake vers DuckDB et dbt-duckdb, sur un univers Boréal redessiné. Ce cours porte encore l'ancienne pile et l'ancienne numérotation : il sert de matière au futur Cours 08, qui n'est pas encore écrit. Les cours 01 à 06 du dossier, eux, sont à jour. Voir README.md.

Où vous êtes : le pipeline complet existe et tourne, on l'entoure enfin d'un filet, et on rend le tout présentable.

En 30 secondes :

- Git pour l'analytique : une branche par changement, une *pull request* par revue. Le SQL se relit comme du code.
- L'intégration continue rejoue vos tests à la place de votre discipline, à chaque proposition de changement.
- Un pipeline de CI dbt tient en deux gestes : `sqlfluff` pour la forme, `dbt build` sur un schéma jetable pour le fond.
- La qualité analytique dépasse les tests : fraîcheur des sources, sévérité graduée, seuils tolérés.
- Le livrable final n'est pas le code, c'est le dépôt : ce qu'un recruteur ouvre en premier, c'est le README.

Comment lire ce module. C'est le capstone. Il n'ajoute presque rien au pipeline (celui-ci est complet depuis le Cours 06) et beaucoup autour : ce qui garantit qu'il reste correct quand plusieurs personnes y touchent, et ce qui le rend montrable. Les deux moitiés se tiennent. Une plateforme sans filet finit par casser en silence ; une plateforme irréprochable que personne ne sait lire ne vous sert à rien en entrevue.

Ce module suppose que vous avez un compte GitHub. Tout ce qui suit est gratuit sur un dépôt public : GitHub Actions offre un quota largement suffisant pour un projet de cette taille.

## Objectifs

À la fin de ce module, vous saurez :

- travailler par branche et par *pull request* sur un projet analytique, et dire ce qu'on regarde dans la revue d'un modèle dbt ;
  - expliquer ce qu'est l'intégration continue et ce qu'elle protège, sans confondre avec le déploiement continu ;
  - lire et écrire un workflow GitHub Actions simple : déclencheur, *job*, étapes, secrets ;
  - monter une CI dbt en deux temps — `sqlfluff` pour la forme, `dbt build` sur un schéma de CI jetable pour le fond ;
  - gérer les identifiants d'un pipeline automatisé sans jamais les écrire dans le dépôt, et distinguer un `profiles.yml` de secrets d'un `profiles.yml` de références ;
  - élargir la qualité au-delà des tests : fraîcheur des sources, sévérité, seuils, et ce qu'un test ne dira jamais ;
  - situer honnêtement ce qu'un agent d'IA apporte à une plateforme analytique ;
  - empaqueter le projet en pièce de portfolio.
- 

## Théorie

Partie A — Travailler à plusieurs (sections 1-2). Partie B — Automatiser (sections 3-5) : Actions, la CI dbt, les secrets. Partie C — Aller plus loin que les tests (sections 6-7) : qualité analytique, agents d'IA. Partie D — Livrer (section 8).

### 1. Git pour l'analytique : ce qui change, et ce qui ne change pas

*Labo : Labo\_07.md, étape 8.*

Vous versionnez depuis le Cours 00, mais seul et en ligne droite : tout est allé sur la branche principale. Dès qu'une deuxième personne touche au projet, cette façon de faire ne tient plus.

Le geste de base est la branche : une copie de travail nommée d'après ce qu'on y fait (`ajout-dim-fournisseur`), où l'on commit librement sans déranger la branche principale. Quand c'est prêt, on ouvre une *pull request*, une demande d'intégration : « voici mes changements, regardez-les avant que je les fusionne ». C'est là que la revue a lieu, et c'est là que l'automatisation s'accroche.

Ce qui ne change pas par rapport au génie logiciel : une branche par sujet, des commits qui racontent quelque chose, une revue avant de fusionner. Ce qui change, c'est ce qu'on regarde en revue. Sur un modèle dbt, les questions utiles sont peu nombreuses et toujours les mêmes.

| On vérifie                                                                      | Parce que                                                    |
|---------------------------------------------------------------------------------|--------------------------------------------------------------|
| Le grain est déclaré et prouvé par un test                                      | c'est l'erreur la plus coûteuse et la plus silencieuse       |
| Aucun nom de table en dur — <code>ref()</code> et <code>source()</code> partout | sinon le graphe, l'ordre d'exécution et le lignage sont faux |
| La logique métier n'est pas dans le staging                                     | une couche non réutilisable est une couche à réécrire        |
| Le modèle a une description                                                     | une doc écrite plus tard n'est jamais écrite                 |
| Les tests nouveaux couvrent le changement                                       | un modèle sans test est une promesse sans preuve             |
| Aucun secret, aucun identifiant personnel                                       | ils restent dans l'historique même après suppression         |

Piège — la revue qui ne regarde que le SQL. Le réflexe est de relire la requête et de dire « ça a l'air bon ». Or ce qui casse une plateforme analytique n'est presque jamais une faute de SQL : c'est un grain non déclaré, une jointure qui duplique, une règle métier posée au mauvais étage. La revue porte sur les décisions de modélisation, pas sur la syntaxe : celle-ci, la machine s'en charge (section 4).

## 2. Intégration continue : rejouer vos tests à votre place

*Labo : Labo\_07.md, étape 5.*

L'intégration continue, ou CI, est le fait de rejouer automatiquement un ensemble de vérifications à chaque proposition de changement. Rien de plus. Elle n'invente aucun test : elle lance ceux que vous avez déjà écrits, systématiquement, y compris le vendredi à 17 h.

Le déploiement continu, ou CD, est l'étape d'après : pousser automatiquement en production ce qui a passé la CI. Les deux lettres voyagent ensemble mais ne se valent pas, beaucoup d'équipes matures ont une CI stricte et un déploiement volontairement manuel. Sur un projet analytique, c'est souvent le bon réglage : on veut que la production se reconstruise quand *on* le décide, pas au moment où une revue est approuvée.

Ce que la CI protège vraiment. Pas votre code, votre attention. Vous savez qu'il faut lancer `dbt build` avant de fusionner ; vous le ferez neuf fois sur dix. La dixième est celle où le test de grain aurait attrapé un doublon. La CI transforme une discipline en garantie, et c'est la raison pour laquelle un projet analytique sans CI finit toujours par livrer une donnée fausse.

Sur un projet dbt, la CI a deux natures de vérification, et il faut les distinguer.

La forme : le SQL est-il écrit selon les conventions du projet ? C'est rapide, ça ne touche pas l'entrepôt, et ça évite les débats de style en revue. L'outil est `sqlfluff`.

Le fond : le projet se construit-il, et les tests passent-ils ? Ça exige une vraie connexion à Snowflake et un endroit où écrire. C'est `dbt build`.

## 3. GitHub Actions : le vocabulaire minimal

*Labo : Labo\_07.md, étape 4.*

GitHub Actions exécute des scripts en réponse à des événements sur le dépôt. Cinq mots suffisent à lire n'importe quel workflow.

| Mot               | Ce que c'est                                                                                        |
|-------------------|-----------------------------------------------------------------------------------------------------|
| workflow          | un fichier <code>.yaml</code> dans <code>.github/workflows/</code> , décrivant quoi lancer et quand |
| déclencheur (on:) | l'événement qui lance le workflow — ici, l'ouverture d'une <i>pull request</i>                      |
| job               | un bloc de travail, exécuté sur une machine neuve                                                   |
| runner            | cette machine jetable, fournie par GitHub — souvent <code>ubuntu-latest</code>                      |
| étape (steps:)    | une commande ou une action réutilisable, exécutée dans l'ordre                                      |

Le point important, et celui qu'on oublie : le *runner* est neuf à chaque exécution. Il ne connaît ni votre poste, ni votre environnement conda, ni votre `~/dbt/profiles.yml`. Tout ce dont le *job* a besoin doit être installé et fourni dans le workflow, c'est ce qui rend ces fichiers verbeux, et c'est aussi ce qui garantit que ça marchera chez le prochain qui clone le dépôt.

#### 4. La CI d'un projet dbt : deux gestes

*Labo : Labo\_07.md, étape 2.*

Le premier geste ne coûte rien. `sqlfluff` lit vos fichiers `.sql`, comprend la Jinja de dbt, et signale les écarts aux conventions : mots-clés en minuscules, indentation, virgules en tête ou en fin de ligne, alias explicites. On le configure une fois dans `.sqlfluff` (dialecte `snowflake`, gabarit `dbt`) et on ne rediscute plus jamais du style. `sqlfluff lint` signale ; `sqlfluff fix` corrige la plupart des écarts tout seul.

Le second geste est le vrai filet : lancer `dbt build` sur les modèles de la *pull request*. Il faut donc que la CI puisse écrire quelque part dans Snowflake, et surtout pas dans les schémas de production.

Pourquoi un schéma de CI jetable. Si la CI construisait dans `MARTS`, une *pull request* cassée abîmerait la production avant même d'être relue, exactement ce qu'on cherche à empêcher. On lui donne donc sa propre cible, écrivant dans des schémas préfixés (`CI_STAGING`, `CI_MARTS`). C'est la même macro `generate_schema_name` que depuis le Cours 03 qui s'en charge : la CI est juste un troisième environnement, à côté de dev et prod.

Reste une question de coût. Reconstruire tout le projet à chaque *pull request* est acceptable sur Boréal (quelques minutes), mais devient absurde sur un projet de mille modèles. La réponse de dbt s'appelle la CI mince (*slim CI*) : en comparant l'état courant à celui de la dernière exécution de production, dbt ne reconstruit que ce qui a changé et ce qui en dépend, avec `dbt build --select state:modified+`. On le nomme parce que le terme revient partout ; on ne l'installe pas ici, car il exige de conserver et de publier les artefacts de la production.

#### 5. Les secrets d'un pipeline automatisé

*Labo : Labo\_07.md, étape 3.*

La CI doit se connecter à Snowflake, donc disposer d'identifiants, sur une machine publique, dans un dépôt public. C'est le point où l'on se fait avoir.

La règle tenue depuis le Cours 00 ne change pas : aucun secret dans le dépôt. Sur GitHub, les valeurs sensibles se rangent dans les secrets du dépôt (Settings → Secrets and variables → Actions). Le workflow y accède par `{{ secrets.NOM }}`, et GitHub masque la valeur dans les journaux d'exécution.

Une nuance mérite d'être posée franchement, parce qu'elle semble contredire le Cours 00. On avait dit : `profiles.yml` reste hors du dépôt. Pour la CI, on va pourtant en versionner un, mais ce n'est pas le même fichier. Celui de votre poste contient une connexion réelle ; celui de la CI ne contient que des appels `env_var(...)`, c'est-à-dire des noms de variables. Il décrit où chercher les valeurs, il ne les porte pas.

|           | <code>~/dbt/profiles.yml</code> (votre poste)                 | <code>ci/profiles.yml</code> (le dépôt)  |
|-----------|---------------------------------------------------------------|------------------------------------------|
| Contient  | une connexion réelle, un mot de passe en <code>env_var</code> | uniquement des <code>env_var(...)</code> |
| Versionné | jamais                                                        | oui — il n'y a rien à protéger dedans    |
| Cible     | dev, prod                                                     | ci                                       |

Piège — le secret qui reste dans l'historique. Un mot de passe committé puis supprimé au commit suivant est toujours là : Git garde tout. Le retirer exige de réécrire l'historique, et sur un dépôt déjà public il faut de toute façon considérer le secret comme brûlé et le changer. D'où la seule règle qui tienne : vérifier avant le premier `git push`, jamais après.

## 6. La qualité analytique au-delà des tests

*Labo : Labo\_07.md, étapes 1 et 6.*

Les tests dbt du Cours 04 répondent à « la donnée est-elle conforme ? ». Trois angles morts demandent autre chose.

La fraîcheur. Un test `not_null` passe parfaitement sur une table qui n'a pas bougé depuis trois semaines. dbt sait vérifier l'âge d'une source (`dbt source freshness`) à condition qu'on déclare, dans le `.yml` de la source, quelle colonne porte l'horodatage de chargement et à partir de quel délai on s'inquiète. C'est souvent le test le plus utile d'un projet, et le dernier écrit.

La sévérité. Tous les manquements ne se valent pas. Une clé de dimension en double doit arrêter le pipeline ; trois adresses mal formatées méritent un avertissement, pas un blocage. dbt permet de régler `severity` : `warn` sur un test, et de fixer des seuils (`error_if`, `warn_if`) : « alerte à partir de 10 lignes fautives, échec à partir de 100 ». Sans ce réglage, une équipe finit par désactiver les tests qui crient trop, et perd du même coup ceux qui comptent.

Le contrat. Un modèle consommé par d'autres peut déclarer un contrat : la liste de ses colonnes et de leurs types, que dbt fait respecter à la construction. Renommer une colonne devient alors une rupture explicite plutôt qu'une surprise en aval. C'est le pendant, côté données, de la revue de la section 1.

Ce qu'un test ne dira jamais. Il ne dit pas si vous avez modélisé le bon processus métier, si le grain choisi répond à la question posée, ni si la marge que vous calculez est celle que la finance appelle marge. Les tests protègent la cohérence, pas la pertinence. Celle-ci se vérifie en parlant au métier, et c'est la partie du travail qu'aucun outil ne prendra.

## 7. Les agents d'IA sur une plateforme analytique

Le sujet est partout, y compris dans les offres d'emploi en ingénierie analytique. Autant le situer précisément, parce que la question posée en entrevue n'est pas « savez-vous appeler un modèle ? » mais « qu'est-ce que ça change à votre travail ? ».

Ce qui rend une plateforme dbt hospitalière à un agent, c'est qu'elle est déjà entièrement écrite. Chaque exécution produit un `manifest.json` : le catalogue complet du projet, les modèles, leurs colonnes, leurs dépendances, leurs tests, leurs descriptions. Un agent n'a pas à deviner la structure de l'entrepôt, elle lui est servie en texte structuré. C'est une différence de nature avec une base sans couche de transformation, où il faudrait tout inférer.

Trois usages tiennent debout aujourd'hui :

| Usage                       | Ce que l'agent fait                                                                                       | Ce qui reste humain                                    |
|-----------------------------|-----------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Documenter                  | proposer les <code>description</code> : manquantes à partir du SQL et des noms de colonnes                | valider : une description inventée est pire qu'absente |
| Relire                      | signaler en <i>pull request</i> un <code>ref()</code> manquant, un modèle sans test, un grain non déclaré | décider si le modèle est le bon                        |
| Répondre en langage naturel | traduire une question métier en requête sur les marts, en s'appuyant sur le lignage                       | vérifier le chiffre avant de le diffuser               |

Et une limite qu'il faut savoir énoncer : un agent est d'autant plus utile que la plateforme est déjà bien tenue. Il lit vos descriptions, vos tests, votre lignage, sur un projet sans grain déclaré ni documentation, il produit des réponses plausibles et fausses, avec l'aplomb en plus. L'ordre des opérations n'est donc pas « ajouter de l'IA à la plateforme » mais « rendre la plateforme lisible, ce qui la rend exploitable par un agent ». Les six cours précédents étaient ce travail-là.

Position de cette formation. On s'arrête à la compréhension. Bâtir un agent est un projet en soi (clé d'API, coûts, garde-fous, évaluation des réponses) et le faire à moitié dans un module de fin donnerait une démonstration de vitrine. Ce qui est défendable en entrevue, c'est d'expliquer pourquoi `manifest.json` change la donne et où l'humain reste indispensable.

8. Le livrable : un dépôt qu'on peut lire

Labo : `Labo_07.md`, étapes 7 et 8.

Le projet est fini ; il reste à le rendre lisible par quelqu'un qui n'a pas vécu les sept modules. C'est un travail court et rentable, le README est la seule chose que la plupart des visiteurs liront.

Ce qu'un lecteur technique cherche, dans l'ordre : ce que fait le projet, sur quelle pile, à quoi ça ressemble, comment le rejouer, et ce que vous en avez appris. Une image aide plus qu'un paragraphe : le graphe de lignage de `dbt docs` et une page du rapport Power BI disent en deux captures ce que trois écrans de texte n'établissent pas.

Piège — le README par défaut. `dbt init` écrit un README générique, en anglais, qui parle du projet de démonstration de dbt. Le laisser en place est le signal le plus visible d'un projet abandonné. C'est la première chose à remplacer.

Un dernier arbitrage, sur la reproductibilité. Le brut de Boréal n'est pas versionné, ce sont des données, et le `.gitignore` les exclut depuis le Cours 00. Un visiteur ne peut donc pas rejouer le projet tel quel, sauf si le générateur qui fabrique ces données accompagne le dépôt. Copier `generer_sources.py` dans le dépôt du projet est un geste d'emballage assumé : il rend la pièce autonome, au prix d'un deuxième exemplaire d'un fichier dont l'original vit avec la formation. C'est un choix, pas une évidence, et savoir l'expliquer vaut mieux que de le trancher au hasard.

---

## Points clés à retenir

- Une branche par sujet, une *pull request* par revue. En revue d'un modèle dbt, on regarde le grain, les `ref()`, l'étage de la logique métier, la description, les tests, l'absence de secret.
  - La CI rejoue vos vérifications à chaque proposition de changement ; le CD déploie ce qui a passé. Les deux sont indépendants, et un déploiement manuel reste défendable.
  - Un workflow GitHub Actions tourne sur un *runner* neuf : tout ce dont il a besoin doit être installé dans le fichier.
  - La CI dbt tient en deux gestes : `sqlfluff` pour la forme (sans toucher l'entrepôt), `dbt build` pour le fond, sur une cible `ci` qui écrit dans des schémas jetables, jamais dans la production.
  - Les secrets vivent dans les secrets GitHub. Un `profiles.yml` de CI ne contient que des `env_var(...)` : il se versionne, parce qu'il ne porte aucune valeur.
  - Au-delà des tests : `dbt source freshness` pour l'âge des données, `severity` et seuils pour graduer, les contrats pour figer une interface. Les tests protègent la cohérence, pas la pertinence.
  - Un agent d'IA exploite le `manifest.json` de dbt — un catalogue déjà écrit. Il documente, relit, traduit des questions ; il ne décide pas du grain. Il est d'autant plus utile que la plateforme est déjà bien tenue.
  - Le livrable final est le dépôt. Le README par défaut de `dbt init` est le premier signal à effacer.
- 

## Exercices

Les exercices de ce cours vivent dans `Exercices_07.md`, et leur corrigé dans `Corrige_07.md`. Faites-les après le labo : plusieurs énoncés supposent que la manipulation a été faite.